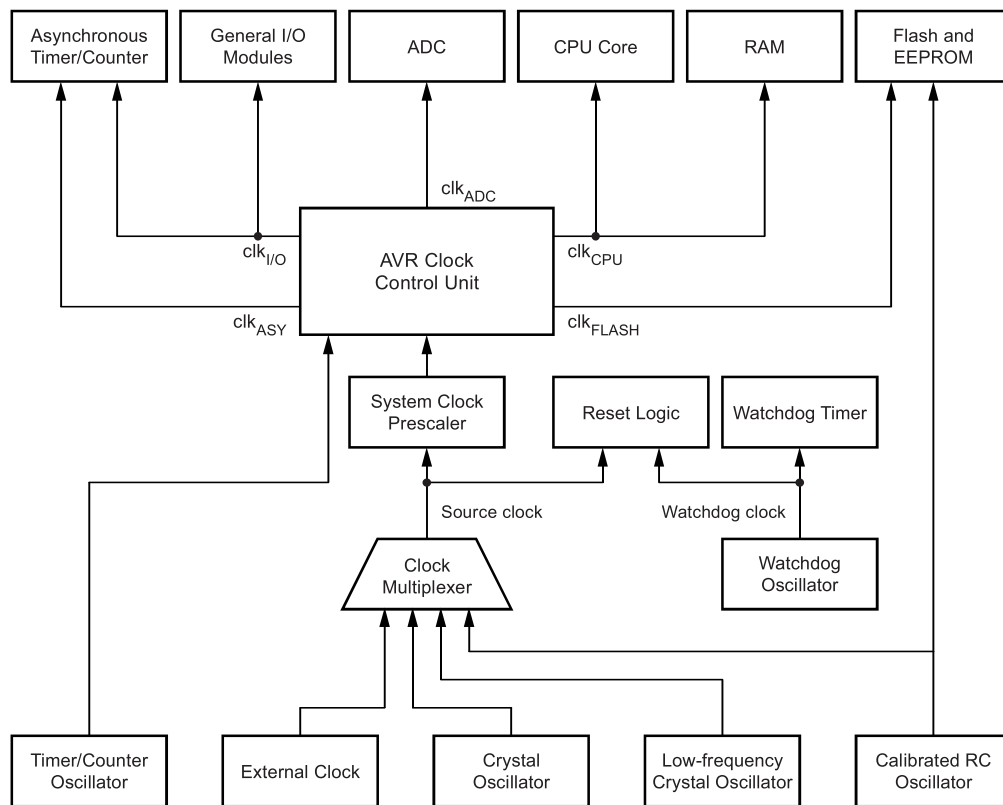


8. System Clock and Clock Options

8.1 Clock Systems and their Distribution

Figure 8-1 presents the principal clock systems in the AVR® and their distribution. All of the clocks need not be active at a given time. In order to reduce power consumption, the clocks to modules not being used can be halted by using different sleep modes, as described in Section 9. “Power Management and Sleep Modes” on page 34. The clock systems are detailed below.

Figure 8-1. Clock Distribution



8.1.1 CPU Clock – clk_{CPU}

The CPU clock is routed to parts of the system concerned with operation of the AVR core. Examples of such modules are the general purpose register file, the status register and the data memory holding the stack pointer. Halting the CPU clock inhibits the core from performing general operations and calculations.

8.1.2 I/O Clock – $clk_{I/O}$

The I/O clock is used by the majority of the I/O modules, like Timer/Counters, SPI, and USART. The I/O clock is also used by the external interrupt module, but note that some external interrupts are detected by asynchronous logic, allowing such interrupts to be detected even if the I/O clock is halted. Also note that start condition detection in the USI module is carried out asynchronously when $clk_{I/O}$ is halted, TWI address recognition in all sleep modes.

8.1.3 Flash Clock – clk_{FLASH}

The flash clock controls operation of the flash interface. The flash clock is usually active simultaneously with the CPU clock.

10. System Control and Reset

10.1 Resetting the AVR

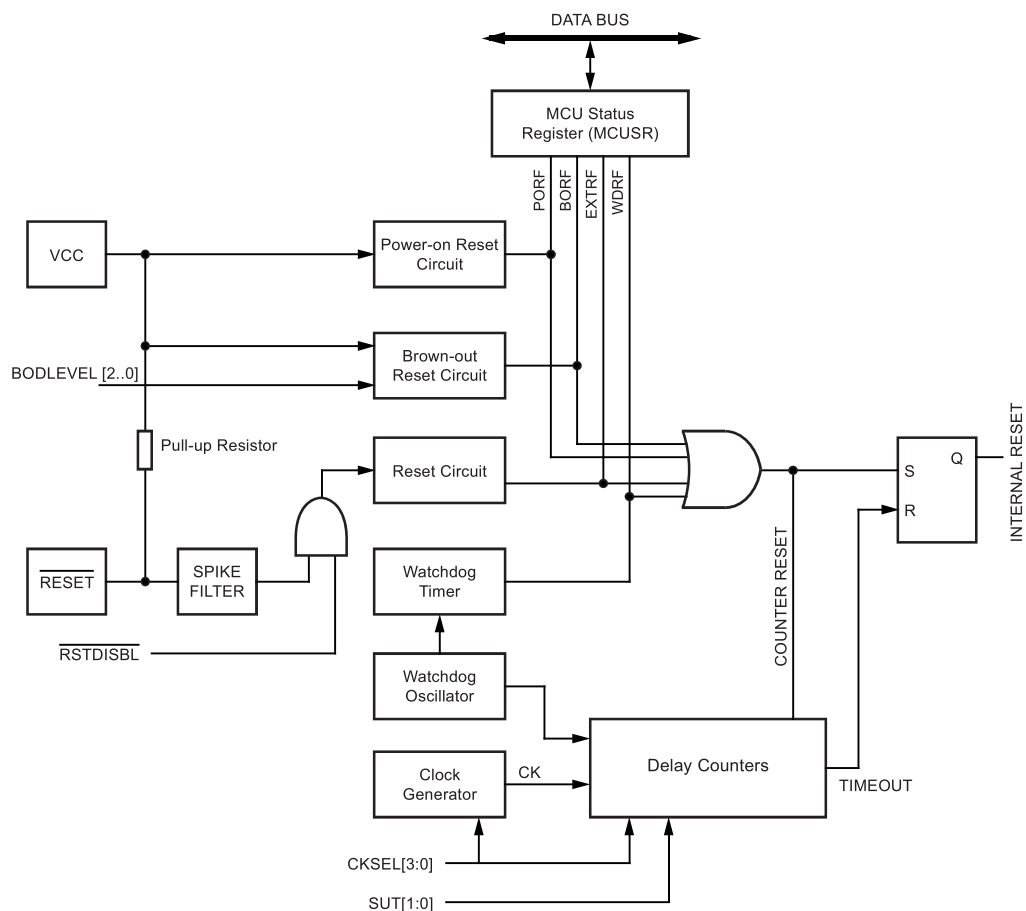
During reset, all I/O registers are set to their initial values, and the program starts execution from the reset vector. For the Atmel® ATmega328P, the instruction placed at the reset vector must be an RJMP – relative jump – instruction to the reset handling routine. If the program never enables an interrupt source, the interrupt vectors are not used, and regular program code can be placed at these locations. This is also the case if the reset vector is in the application section while the interrupt vectors are in the boot section. The circuit diagram in [Figure 10-1](#) shows the reset logic. [Table 28-4 on page 261](#) defines the electrical parameters of the reset circuitry. The I/O ports of the AVR® are immediately reset to their initial state when a reset source goes active. This does not require any clock source to be running. After all reset sources have gone inactive, a delay counter is invoked, stretching the internal reset. This allows the power to reach a stable level before normal operation starts. The time-out period of the delay counter is defined by the user through the SUT and CKSEL fuses. The different selections for the delay period are presented in [Section 8.2 “Clock Sources” on page 25](#).

10.2 Reset Sources

The Atmel ATmega328P has four sources of reset:

- Power-on reset. The MCU is reset when the supply voltage is below the power-on reset threshold (V_{POT}).
- External reset. The MCU is reset when a low level is present on the $\overline{RESET}$ pin for longer than the minimum pulse length.
- Watchdog system reset. The MCU is reset when the watchdog timer period expires and the watchdog system reset mode is enabled.
- Brown-out reset. The MCU is reset when the supply voltage V_{CC} is below the brown-out reset threshold (V_{BOT}) and the brown-out detector is enabled.

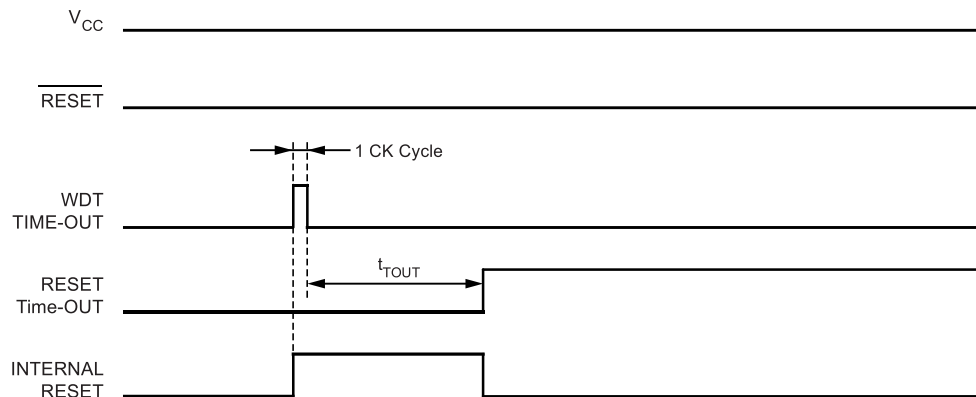
Figure 10-1. Reset Logic



10.6 Watchdog System Reset

When the watchdog times out, it will generate a short reset pulse of one CK cycle duration. On the falling edge of this pulse, the delay timer starts counting the time-out period t_{TOUT} . Refer to [Section 10.8 “Watchdog Timer” on page 43](#) for details on operation of the watchdog timer.

Figure 10-6. Watchdog System Reset During Operation



10.7 Internal Voltage Reference

Atmel® ATmega328P features an internal bandgap reference. This reference is used for brown-out detection, and it can be used as an input to the analog comparator or the ADC.

10.7.1 Voltage Reference Enable Signals and Start-up Time

The voltage reference has a start-up time that may influence the way it should be used. The start-up time is given in [Section 28.6 “System and Reset Characteristics” on page 261](#). To save power, the reference is not always turned on. The reference is on during the following situations:

1. When the BOD is enabled (by programming the BODLEVEL [2:0] Fuses).
2. When the bandgap reference is connected to the analog comparator (by setting the ACBG bit in ACSR).
3. When the ADC is enabled.

Thus, when the BOD is not enabled, after setting the ACBG bit or enabling the ADC, the user must always allow the reference to start up before the output from the analog comparator or ADC is used. To reduce power consumption in power-down mode, the user can avoid the three conditions above to ensure that the reference is turned off before entering power-down mode.

10.8 Watchdog Timer

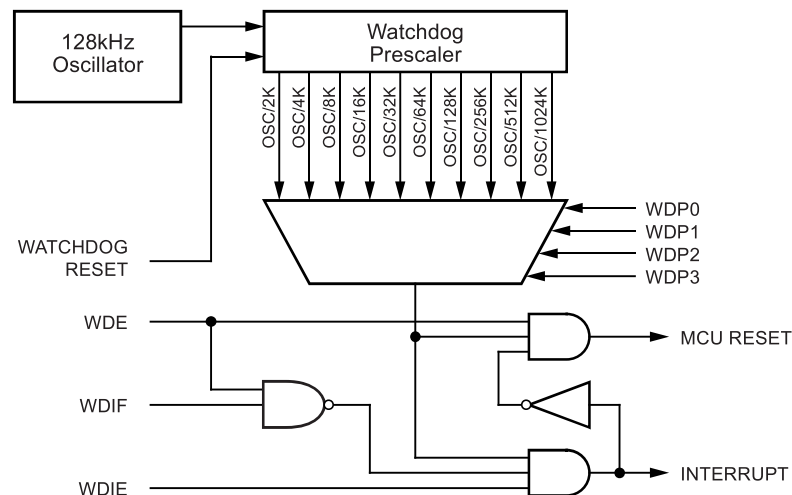
10.8.1 Features

- Clocked from separate on-chip oscillator
- 3 operating modes
 - Interrupt
 - System reset
 - Interrupt and system reset
- Selectable time-out period from 16ms to 8s
- Possible hardware fuse watchdog always on (WDTON) for fail-safe mode

10.8.2 Overview

Atmel® ATmega328P has an enhanced watchdog timer (WDT). The WDT is a timer counting cycles of a separate on-chip 128kHz oscillator. The WDT gives an interrupt or a system reset when the counter reaches a given time-out value. In normal operation mode, it is required that the system uses the WDR - watchdog timer reset - instruction to restart the counter before the time-out value is reached. If the system doesn't restart the counter, an interrupt or system reset will be issued.

Figure 10-7. Watchdog Timer



In interrupt mode, the WDT gives an interrupt when the timer expires. This interrupt can be used to wake the device from sleep-modes, and also as a general system timer. One example is to limit the maximum time allowed for certain operations, giving an interrupt when the operation has run longer than expected. In system reset mode, the WDT gives a reset when the timer expires. This is typically used to prevent system hang-up in case of runaway code. The third mode, Interrupt and system reset mode, combines the other two modes by first giving an interrupt and then switch to system reset mode. This mode will for instance allow a safe shutdown by saving critical parameters before a system reset.

The watchdog always on (WDTON) fuse, if programmed, will force the watchdog timer to system reset mode. With the fuse programmed the system reset mode bit (WDE) and interrupt mode bit (WDIE) are locked to 1 and 0 respectively. To further ensure program security, alterations to the watchdog set-up must follow timed sequences.

The sequence for clearing WDE and changing time-out configuration is as follows:

1. In the same operation, write a logic one to the watchdog change enable bit (WDCE) and WDE. A logic one must be written to WDE regardless of the previous value of the WDE bit.
2. Within the next four clock cycles, write the WDE and watchdog prescaler bits (WDP) as desired, but with the WDCE bit cleared. This must be done in one operation.

The following code example shows one assembly and one C function for turning off the watchdog timer. The example assumes that interrupts are controlled (e.g. by disabling interrupts globally) so that no interrupts will occur during the execution of these functions.

Assembly Code Example ⁽¹⁾
<pre> WDT_off: ; Turn off global interrupt cli ; Reset Watchdog Timer wdr ; Clear WDRF in MCUSR in r16, MCUSR andi r16, (0xff & (0<<WDRF)) out MCUSR, r16 ; Write logical one to WDCE and WDE ; Keep old prescaler setting to prevent unintentional time-out lds r16, WDTCSR ori r16, (1<<WDCE) (1<<WDE) sts WDTCSR, r16 ; Turn off WDT ldi r16, (0<<WDE) sts WDTCSR, r16 ; Turn on global interrupt sei ret </pre>
C Code Example ⁽¹⁾
<pre> void WDT_off(void) { __disable_interrupt(); __watchdog_reset(); /* Clear WDRF in MCUSR */ MCUSR &= ~(1<<WDRF); /* Write logical one to WDCE and WDE */ /* Keep old prescaler setting to prevent unintentional time-out */ WDTCSR = (1<<WDCE) (1<<WDE); /* Turn off WDT */ WDTCSR = 0x00; __enable_interrupt(); } </pre>

- Note:
1. See [Section 5. “About Code Examples” on page 8](#).
 2. Note: If the watchdog is accidentally enabled, for example by a runaway pointer or brown-out condition, the device will be reset and the watchdog timer will stay enabled. If the code is not set up to handle the watchdog, this might lead to an eternal loop of time-out resets. To avoid this situation, the application software should always clear the watchdog system reset flag (WDRF) and the WDE control bit in the initialization routine, even if the watchdog is not in use.

11. Interrupts

This section describes the specifics of the interrupt handling as performed in Atmel® ATmega328P. For a general explanation of the AVR® interrupt handling, refer to [Section 6.7 “Reset and Interrupt Handling” on page 15](#).

- Each interrupt vector occupies two instruction words in Atmel ATmega328P.
- In Atmel ATmega328P, the reset vector is affected by the BOOTRST fuse, and the interrupt vector start address is affected by the IVSEL bit in MCUCR.

11.1 Interrupt Vectors in ATmega328P

Table 11-1. Reset and Interrupt Vectors in ATmega328P

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x0002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 capture event
12	0x0016	TIMER1 COMPA	Timer/Counter1 compare match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 compare match B
14	0x001A	TIMER1 OVF	Timer/Counter1 overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 compare match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 compare match B
17	0x0020	TIMER0 OVF	Timer/Counter0 overflow
18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete
22	0x002A	ADC	ADC conversion complete
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Analog comparator
25	0x0030	TWI	2-wire serial interface
26	0x0032	SPM READY	Store program memory ready

31. Instruction Set Summary

Mnemonics	Operands	Description	Operation	Flags	#Clocks
Arithmetic and Logic Instructions					
ADD	Rd, Rr	Add two registers	$Rd \leftarrow Rd + Rr$	Z,C,N,V,H	1
ADC	Rd, Rr	Add with carry two registers	$Rd \leftarrow Rd + Rr + C$	Z,C,N,V,H	1
ADIW	Rdl, K	Add immediate to word	Rdh: Rdl $\leftarrow$ Rdh: Rdl + K	Z,C,N,V,S	2
SUB	Rd, Rr	Subtract two registers	$Rd \leftarrow Rd - Rr$	Z,C,N,V,H	1
SUBI	Rd, K	Subtract constant from register	$Rd \leftarrow Rd - K$	Z,C,N,V,H	1
SBC	Rd, Rr	Subtract with carry two registers	$Rd \leftarrow Rd - Rr - C$	Z,C,N,V,H	1
SBCI	Rd, K	Subtract with carry constant from reg.	$Rd \leftarrow Rd - K - C$	Z,C,N,V,H	1
SBIW	Rdl, K	Subtract immediate from word	Rdh: Rdl $\leftarrow$ Rdh: Rdl - K	Z,C,N,V,S	2
AND	Rd, Rr	Logical AND registers	$Rd \leftarrow Rd \times Rr$	Z,N,V	1
ANDI	Rd, K	Logical AND register and constant	$Rd \leftarrow Rd \times K$	Z,N,V	1
OR	Rd, Rr	Logical OR registers	$Rd \leftarrow Rd \vee Rr$	Z,N,V	1
ORI	Rd, K	Logical OR register and constant	$Rd \leftarrow Rd \vee K$	Z,N,V	1
EOR	Rd, Rr	Exclusive OR registers	$Rd \leftarrow Rd \oplus Rr$	Z,N,V	1
COM	Rd	One's complement	$Rd \leftarrow 0xFF - Rd$	Z,C,N,V	1
NEG	Rd	Two's complement	$Rd \leftarrow 0x00 - Rd$	Z,C,N,V,H	1
SBR	Rd, K	Set bit(s) in register	$Rd \leftarrow Rd \vee K$	Z,N,V	1
CBR	Rd, K	Clear bit(s) in register	$Rd \leftarrow Rd \times (0xFF - K)$	Z,N,V	1
INC	Rd	Increment	$Rd \leftarrow Rd + 1$	Z,N,V	1
DEC	Rd	Decrement	$Rd \leftarrow Rd - 1$	Z,N,V	1
TST	Rd	Test for zero or minus	$Rd \leftarrow Rd \times Rd$	Z,N,V	1
CLR	Rd	Clear register	$Rd \leftarrow Rd \oplus Rd$	Z,N,V	1
SER	Rd	Set register	$Rd \leftarrow 0xFF$	None	1
MUL	Rd, Rr	Multiply unsigned	R1:R0 $\leftarrow$ Rd $\times$ Rr	Z,C	2
MULS	Rd, Rr	Multiply signed	R1:R0 $\leftarrow$ Rd $\times$ Rr	Z,C	2
MULSU	Rd, Rr	Multiply signed with unsigned	R1:R0 $\leftarrow$ Rd $\times$ Rr	Z,C	2
FMUL	Rd, Rr	Fractional multiply unsigned	R1:R0 $\leftarrow$ (Rd $\times$ Rr) $\ll$ 1	Z,C	2
FMULS	Rd, Rr	Fractional multiply signed	R1:R0 $\leftarrow$ (Rd $\times$ Rr) $\ll$ 1	Z,C	2
FMULSU	Rd, Rr	Fractional multiply signed with unsigned	R1:R0 $\leftarrow$ (Rd $\times$ Rr) $\ll$ 1	Z,C	2
Branch Instructions					
RJMP	k	Relative jump	$PC \leftarrow PC + k + 1$	None	2
IJMP		Indirect jump to (Z)	$PC \leftarrow Z$	None	2
JMP	k	Direct jump	$PC \leftarrow k$	None	3
RCALL	k	Relative subroutine call	$PC \leftarrow PC + k + 1$	None	3
ICALL		Indirect call to (Z)	$PC \leftarrow Z$	None	3
CALL	k	Direct subroutine call	$PC \leftarrow k$	None	4
RET		Subroutine return	$PC \leftarrow \text{STACK}$	None	4
RETI		Interrupt return	$PC \leftarrow \text{STACK}$	I	4
CPSE	Rd, Rr	Compare, skip if equal	if (Rd = Rr) $PC \leftarrow PC + 2$ or 3	None	1/2/3
CP	Rd, Rr	Compare	$Rd - Rr$	Z,N,V,C,H	1

31. Instruction Set Summary (Continued)

Mnemonics	Operands	Description	Operation	Flags	#Clocks
CPC	Rd, Rr	Compare with carry	$Rd - Rr - C$	Z, N, V, C, H	1
CPI	Rd, K	Compare register with immediate	$Rd - K$	Z, N, V, C, H	1
SBRC	Rr, b	Skip if bit in register cleared	if $(Rr(b) = 0)$ $PC \leftarrow PC + 2$ or 3	None	1/2/3
SBRS	Rr, b	Skip if bit in register is set	if $(Rr(b)=1)$ $PC \leftarrow PC + 2$ or 3	None	1/2/3
SBIC	P, b	Skip if bit in I/O register cleared	if $(P(b)=0)$ $PC \leftarrow PC + 2$ or 3	None	1/2/3
SBIS	P, b	Skip if bit in I/O register is set	if $(P(b)=1)$ $PC \leftarrow PC + 2$ or 3	None	1/2/3
BRBS	s, k	Branch if status flag set	if $(SREG(s) = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRBC	s, k	Branch if status flag cleared	if $(SREG(s) = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BREQ	k	Branch if equal	if $(Z = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRNE	k	Branch if not equal	if $(Z = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRCS	k	Branch if carry set	if $(C = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRCC	k	Branch if carry cleared	if $(C = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRSH	k	Branch if same or higher	if $(C = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRLO	k	Branch if lower	if $(C = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRMI	k	Branch if minus	if $(N = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRPL	k	Branch if plus	if $(N = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRGE	k	Branch if greater or equal, signed	if $(N \oplus V = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRLT	k	Branch if less than zero, signed	if $(N \oplus V = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRHS	k	Branch if half carry flag set	if $(H = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRHC	k	Branch if half carry flag cleared	if $(H = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRTS	k	Branch if T flag set	if $(T = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRTC	k	Branch if T flag cleared	if $(T = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRVS	k	Branch if overflow flag is set	if $(V = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRVC	k	Branch if overflow flag is cleared	if $(V = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRIE	k	Branch if interrupt enabled	if $(I = 1)$ then $PC \leftarrow PC + k + 1$	None	1/2
BRID	k	Branch if interrupt disabled	if $(I = 0)$ then $PC \leftarrow PC + k + 1$	None	1/2
Bit and Bit-Test Instructions					
SBI	P, b	Set bit in I/O register	$I/O(P, b) \leftarrow 1$	None	2
CBI	P, b	Clear bit in I/O register	$I/O(P, b) \leftarrow 0$	None	2
LSL	Rd	Logical shift left	$Rd(n+1) \leftarrow Rd(n), Rd(0) \leftarrow 0$	Z, C, N, V	1
LSR	Rd	Logical shift right	$Rd(n) \leftarrow Rd(n+1), Rd(7) \leftarrow 0$	Z, C, N, V	1
ROL	Rd	Rotate left through carry	$Rd(0) \leftarrow C, Rd(n+1) \leftarrow Rd(n), C \leftarrow Rd(7)$	Z, C, N, V	1
ROR	Rd	Rotate right through carry	$Rd(7) \leftarrow C, Rd(n) \leftarrow Rd(n+1), C \leftarrow Rd(0)$	Z, C, N, V	1
ASR	Rd	Arithmetic shift right	$Rd(n) \leftarrow Rd(n+1), n=0..6$	Z, C, N, V	1
SWAP	Rd	Swap nibbles	$Rd(3..0) \leftarrow Rd(7..4), Rd(7..4) \leftarrow Rd(3..0)$	None	1
BSET	s	Flag set	$SREG(s) \leftarrow 1$	SREG(s)	1
BCLR	s	Flag clear	$SREG(s) \leftarrow 0$	SREG(s)	1
BST	Rr, b	Bit store from register to T	$T \leftarrow Rr(b)$	T	1
BLD	Rd, b	Bit load from T to register	$Rd(b) \leftarrow T$	None	1
SEC		Set carry	$C \leftarrow 1$	C	1
CLC		Clear carry	$C \leftarrow 0$	C	1
SEN		Set negative flag	$N \leftarrow 1$	N	1

31. Instruction Set Summary (Continued)

Mnemonics	Operands	Description	Operation	Flags	#Clocks
CLN		Clear negative flag	$N \leftarrow 0$	N	1
SEZ		Set zero flag	$Z \leftarrow 1$	Z	1
CLZ		Clear zero flag	$Z \leftarrow 0$	Z	1
SEI		Global interrupt enable	$I \leftarrow 1$	I	1
CLI		Global interrupt disable	$I \leftarrow 0$	I	1
SES		Set signed test flag	$S \leftarrow 1$	S	1
CLS		Clear signed test flag	$S \leftarrow 0$	S	1
SEV		Set twos complement overflow.	$V \leftarrow 1$	V	1
CLV		Clear twos complement overflow	$V \leftarrow 0$	V	1
SET		Set T in SREG	$T \leftarrow 1$	T	1
CLT		Clear T in SREG	$T \leftarrow 0$	T	1
SEH		Set half carry flag in SREG	$H \leftarrow 1$	H	1
CLH		Clear half carry flag in SREG	$H \leftarrow 0$	H	1
Data Transfer Instructions					
MOV	Rd, Rr	Move between registers	$Rd \leftarrow Rr$	None	1
MOVW	Rd, Rr	Copy register word	$Rd+1:Rd \leftarrow Rr+1:Rr$	None	1
LDI	Rd, K	Load immediate	$Rd \leftarrow K$	None	1
LD	Rd, X	Load indirect	$Rd \leftarrow (X)$	None	2
LD	Rd, X+	Load indirect and post-inc.	$Rd \leftarrow (X), X \leftarrow X + 1$	None	2
LD	Rd, -X	Load indirect and pre-dec.	$X \leftarrow X - 1, Rd \leftarrow (X)$	None	2
LD	Rd, Y	Load indirect	$Rd \leftarrow (Y)$	None	2
LD	Rd, Y+	Load indirect and post-inc.	$Rd \leftarrow (Y), Y \leftarrow Y + 1$	None	2
LD	Rd, -Y	Load indirect and pre-dec.	$Y \leftarrow Y - 1, Rd \leftarrow (Y)$	None	2
LDD	Rd, Y+q	Load indirect with displacement	$Rd \leftarrow (Y + q)$	None	2
LD	Rd, Z	Load indirect	$Rd \leftarrow (Z)$	None	2
LD	Rd, Z+	Load indirect and post-inc.	$Rd \leftarrow (Z), Z \leftarrow Z + 1$	None	2
LD	Rd, -Z	Load indirect and pre-dec.	$Z \leftarrow Z - 1, Rd \leftarrow (Z)$	None	2
LDD	Rd, Z+q	Load indirect with displacement	$Rd \leftarrow (Z + q)$	None	2
LDS	Rd, k	Load direct from SRAM	$Rd \leftarrow (k)$	None	2
ST	X, Rr	Store indirect	$(X) \leftarrow Rr$	None	2
ST	X+, Rr	Store indirect and post-inc.	$(X) \leftarrow Rr, X \leftarrow X + 1$	None	2
ST	-X, Rr	Store indirect and pre-dec.	$X \leftarrow X - 1, (X) \leftarrow Rr$	None	2
ST	Y, Rr	Store indirect	$(Y) \leftarrow Rr$	None	2
ST	Y+, Rr	Store indirect and post-inc.	$(Y) \leftarrow Rr, Y \leftarrow Y + 1$	None	2
ST	-Y, Rr	Store indirect and pre-dec.	$Y \leftarrow Y - 1, (Y) \leftarrow Rr$	None	2
STD	Y+q, Rr	Store indirect with displacement	$(Y + q) \leftarrow Rr$	None	2
ST	Z, Rr	Store indirect	$(Z) \leftarrow Rr$	None	2
ST	Z+, Rr	Store indirect and post-inc.	$(Z) \leftarrow Rr, Z \leftarrow Z + 1$	None	2
ST	-Z, Rr	Store indirect and pre-dec.	$Z \leftarrow Z - 1, (Z) \leftarrow Rr$	None	2
STD	Z+q, Rr	Store indirect with displacement	$(Z + q) \leftarrow Rr$	None	2
STS	k, Rr	Store direct to SRAM	$(k) \leftarrow Rr$	None	2
LPM		Load program memory	$R0 \leftarrow (Z)$	None	3

31. Instruction Set Summary (Continued)

Mnemonics	Operands	Description	Operation	Flags	#Clocks
LPM	Rd, Z	Load program memory	$Rd \leftarrow (Z)$	None	3
LPM	Rd, Z+	Load program memory and post-inc	$Rd \leftarrow (Z), Z \leftarrow Z+1$	None	3
SPM		Store program memory	$(Z) \leftarrow R1:R0$	None	–
IN	Rd, P	In port	$Rd \leftarrow P$	None	1
OUT	P, Rr	Out port	$P \leftarrow Rr$	None	1
PUSH	Rr	Push register on stack	$STACK \leftarrow Rr$	None	2
POP	Rd	Pop register from stack	$Rd \leftarrow STACK$	None	2
MCU Control Instructions					
NOP		No operation		None	1
SLEEP		Sleep	(see specific descr. for sleep function)	None	1
WDR		Watchdog reset	(see specific descr. for WDR/timer)	None	1
BREAK		Break	For on-chip debug only	None	N/A